



Elektrotehnički fakultet
Univerzitet u Banjoj Luci

IZVJEŠTAJ PROJEKTOG ZADATAKA

iz predmeta

SISTEMI ZA DIGITALNU OBRADU SIGNALA

Student:
Boško Branković, 1180/21

Mentori:
prof. dr Mitar Simić
prof. dr Mladen Knežić
dipl. inž. Dimitrije Obradović
dipl.inž. Vedran Jovanović
Januar 2026. godine

1. Opis projektnog zadatka

U sklopu projektnog zadatka je potrebno realizovati sistem za kompresiju *grayscale* slika na ADSP-21489 korištenjem diskretne kosinusne transformacije. Kompresija treba da bude bazirana na JPEG standardu za kompresiju.

2. Izrada projektnog zadatka

Projektni zadatak je realizovan korištenjem pomoćnih *python* skripti i alata "CrossCore Embedded studio" verzije 2.12.1. Proces izrade je podijeljen na predprocesiranje, glavno procesiranje i postprocesiranje. Glavno procesiranje se izvršava na ploči ADSP-21489, SHARC procesora.

1. Predprocesiranje

U ovom koraku se ulazni podaci pripremaju za obradu na ploči. Korištenjem *python* skripte *generate_header.py* se, iz foldera zadanog putem argumenata komandne linije (podrazumijevano folder Images/) , za svaku sliku (.bmp ili .jpg formata) generiše fajl zaglavlja u kojem se nalaze osnovni podaci o novogenerisanoj slici poput visine, širine i piksela. Važno je za napomenuti da prethodno pomenuta *python* skripta vrši pretvaranje piksela slike u *grayscale* te ih kao takve smješta u unaprijed definisan jednodimenzionalni niz *data1* tipa *static unsigned char*. Skripta je napisana tako da niz *data1* smjesti u spoljnu memoriju imena *mem_sram*.

Prilikom programiranja potrebno je samo uključiti zaglavlje te se promjenljive unutar istog mogu nesmetano koristiti. Sva zaglavlja su odmah po generisanju smještena u *Debug/* folder.

2. Glavno procesiranje

Nakon što su ulazni podaci pripremljeni, prelazi se na glavno procesiranje. U ovom koraku se koristi 'CrossCore Embedded Studio 2.12.1'. Glavni program, pisan u C programskom jeziku, je razdvojen na nekoliko funkcionalnih jedinica:

- Segmentacija

Ulazne podatke iz niza *data1* razdvajamo u blokove veličine 8x8 te ih kao takve smiještamo u jednodimenzionalni niz *data2*. Svaki blok *N* počinje na memorijskoj lokaciji $\langle \text{početak niza } data2 \rangle + N * 64$. Ukoliko dimenzije slike nisu djeljive sa 8, blokovi koji ne ispunjavaju ove uslove se proširuju sa poslednjim pikselima iz njihovog poslednjeg reda (ili kolone). Upravo ovaj uslov je prepreka kompajleru za korištenje SIMD instrukcija što značajno usporava proces segmentiranja. Bez tog uslova, kompajler bi lako mogao iskoristiti SIMD instrukcije čime bi se procesiralo više piksela odjednom.

- Centriranje oko nule

Nakon što smo smjestili segmentirane podatke u novi niz, kroz isti prolazimo čitajući blok po blok i izvršavamo set operacija nad njima. Prva u nizu operacija jeste centriranje oko nule. U ovom koraku je potrebno jednostavno od svakog piksela oduzeti 128. Jasno je da se novonastali pikseli više ne mogu smjestiti u promjenljivu neoznačenog tipa, s toga izračunavanje u mjestu se vrši nad *current_block* nizom koji je tipa *signed short pm**. Ovaj niz je smješten u programsku memoriju kako bi procesor mogao simultano vršiti *fetch* operaciju nad dva podatka (jedan preko PM magistrale a drugi preko DM magistrale). Važno je za napomenuti da obe magistrale imaju svoje dijelove memorije kojima mogu pristupati te da je potrebno uskladiti lokaciju promjenljive u memoriji sa magistralom koja ju obrađuje (pm/dm ključna riječ pri deklaraciji promjenljive).

- Diskretna kosinusna transformacija (2D DCT)

Sa prethodne dvije funkcije smo pripremili blok za vršenje diskretne kosinusne transformacije. Uz pomoć ove funkcije transformišemo podatke iz vremenskog domena u frekvencijski domen, pri čemu su značajne informacije (nakon vršenja DCT) koncentrisane u gornjem lijevom uglu svakog bloka. Novonastala matrica sadrži tzv. DCT koeficijente. Ona nam pokazuje doprinos svake frekvencije originalnoj slici, pri čemu veći koeficijent označava veći doprinos a manji koeficijent, manji doprinos.

Visoke frekvencije su odstranjene (nulirane) pri čemu se znatno dobija na kompresiji. Opšta formula za dobijanje DCT koeficijenata je:

U rješenju dostupnom na github repozitorijumu, DCT je implementirana na svaki blok zasebno, računajući DCT prvo po redovima, zatim po kolonama. Na ovaj način se svaki red 8x8 bloka tretira kao jedinstven 1D vektor. Ovime je složenost svedena na niži stepen (kubna složenost).

$$F(u, v) = \alpha(u)\alpha(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \cos\left(\frac{\pi}{N} \left(x + \frac{1}{2}\right)u\right) \cos\left(\frac{\pi}{N} \left(y + \frac{1}{2}\right)v\right),$$

1. DCT tipa 2

- Kvantizacija

Nakon što smo izračunali doprinos visokih i niskih frekvencija originalnoj slici, vrši se kvantizacija. U ovom koraku se izvršava kompresija slika, a od matrice kvantizacije zavisi kolika će kompresija biti. Kvantizacija se vrši tako što svaki element bloka dijelimo sa unaprijed definisanom vrijednošću.

- Zig-Zag

U ovom koraku se preraspodjeljuju kvantizovane vrijednosti kako bi se grupisale značajne frekvencije i frekvencije manjeg značaja. Visoke frekvencije su dominantne te su većinski nulirane, zbog čega je od izuzetne važnosti grupisanje istih na desni dio niza (dominantne frekvencije su na početku) kako bi se podaci pripremili za *Run-Length* kodovanje.

- RLE

RLE (*Run-Length Encoding*) je proces kreiranja RL parova koji olakšavaju čitanje i kodovanje istih na osnovu Hafmanovog stabla (više informacije u narednom koraku). U principu, RLE je jednostavan oblik kompresije podataka bez gubitaka koji se zasniva na zamjenjivanju uzastopnih ponavljanja istog simbola jednim parom vrednosti. Prvi broj označava koliko nula treba preskočiti prije nego što se naiđe na broj koji nije nula, a drugi broj je sama vrijednost tog koeficijenta. Na ovaj način, dugački nizovi nula,

koji bi inače zauzimali desetine bajtova, bivaju sažeti u svega par bajtova, dok se čitav ostatak bloka popunjen nulama zamjenjuje jednim jedinim EOB (End of Block) markerom koji signalizira kraj obrade tog bloka.

- Hafmanovo kodovanje

Hafmanovo kodovanje (Huffman Coding) je algoritam za kompresiju podataka bez gubitaka koji se zasniva na principu promjenljive dužine koda. Glavna ideja jeste da simboli koji se u podacima pojavljuju češće dobijaju kraće binarne kodove, dok oni koji se pojavljuju rjeđe dobijaju duže kodove. U kodu na github repozitorijumu je korištena unaprijed izračunata tabela Hafmanovih kodova za AC i DC koeficijente, a RL parovi (njihovi dijelovi) se mapiraju na vrijednosti prethodno pomenute strukture. Važno je za napomenuti da se bitovi u izlazni fajl moraju upisivati jedan za drugim, bez ikakvog poravnanja (padding-a).

- Serijalizacija

Nakon što se prethodne operacije izvrše nad svakim blokom podataka, u izlazni fajl se binarno upisuju markeri i značajni podaci za kreiranje kompresovane slike. Neki od značajnih podataka su kvantizaciona tabela, dužine AC kodova te dužine DC kodova kao i njihovi simboli, veličina slike te niz podataka dobijenih Hafmanovim kodovanjem. Ukoliko niz podataka dobijenih Hafmanovim kodovanjem nije poravnat na jedan bajt, potrebno je dopuniti isti jedinicama. Takođe, ovaj niz podataka ne smije sadržati u sebi 0xFF bajtove bez automatskog praćenja sa 0x00. Razlog za ovo jeste to što svaki marker u .jpeg formatu počinje sa 0xFF.

3. Postprocesiranje

Prilikom upisivanja bajtova u izlazni fajl stiče se utisak da su oni upisani bajt po bajt. Međutim, uzimajući u obzir da je SHARC procesor *word addressable* tj da može adresirati najmanje jednu riječ u memoriji na kojoj je otvoren izlazni fajl, postoji potreba da se nakon dobijanja kompresovanog fajla iz istog pročita i u zaseban fajl upiše njegov svaki četvrti bajt. Na ovaj način se zaobilazi *padding* koji je SHARC procesor uveo u kompresiju jer se postupak postprocesiranja izvršava na računaru koji može adresirati i jedan bajt. Skripta koja vrši čitanje svakog četvrtog bajta i upisivanje istih u novi fajl jeste

read_image.py. Ona kao ulazne argumente prima kompresovanu sliku (ulazni fajl) i izlaznu sliku (izlazni fajl). Rezultat pokretanja prethodno pomenute skripte je ispravna .jpeg slika koja se može otvoriti u proizvoljnom editoru.

Kako bi se dobila dekompresovana slika potrebno je izvršiti *decompressing_tool.py* nad izlaznim .jpeg fajlom te je, pored pomenutog fajla, potrebno specificirati fajl u koji će biti smještena dekompresovana slika. Navedeni fajlovi se specificiraju kao argumenti prilikom pokretanja skripte. Ova skripta će izvršiti inverzni proces kompresije, prateći korak po korak sve operacije koji su pomenute u sekciji glavnog procesiranja. Ona će učitati kompresovanu sliku te iz njenog zaglavlja pročitati sve značajne markere i podatke koji dolaze nakon tih markera a bez kojih rekonstrukcija slike nije moguća. Čitav proces je čisto reverzibilan osim dekvantizacije u kojoj se gubi određena količina podataka.

Upoređivanje dekompresovane i kompresovane slike se vrši pomoću *MSE_calculation.py* skripte koja kao izlaz proizvodi mapu greške u .png formatu.

MSE (Mean Squared Error), ili srednjekvadratna greška, predstavlja numeričko mjerilo razlike između originalne i dekompresovane slike, a izračunava se tako što se za svaki piksel odredi razlika u vrijednosti (greška), ta razlika se kvadrira kako bi se eliminisali negativni znaci i naglasila veća odstupanja, a zatim se izračuna prosjek tih kvadrata za cijelu sliku. Osnovni razlog za ovo računanje je kvantifikacija gubitka informacija koji nastaje tokom JPEG kompresije, čime dobijamo objektivni pokazatelj koliko je proces kvantizacije "pokvario" originalni signal. Rezultati MSE prvenstveno zavise od faktora kvantizacije — što su vrijednosti u tabeli kvantizacije veće, to je kompresija jača, ali je i MSE veći (lošiji kvalitet) — mada na rezultat utiče i sama priroda slike, gde visokofrekventni detalji i oštre ivice obično generišu veće greške u odnosu na glatke površine.

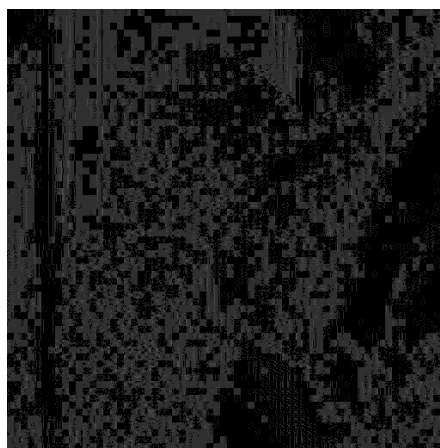
Na slikama ispod su date kompresovana i dekompresovana slika kao i mapa greške:



2. Kompresovana slika



3. Dekompresovana slika



4. Mapa greške

Mapa greške nastaje tako što se pronade razlika između svakog piksela kompresovane i

dekompresovane slike, pri čemu se njeni svjetliji dijelovi (ili specifične boje) obično grupišu oko oštih ivica i tekstura, jer kvantizacija najviše pogađa visoke frekvencije koje su odgovorne za detalje, dok tamni dijelovi mape ukazuju na oblasti gde je rekonstrukcija gotovo savršena. Ukoliko se na mapi uoči pravilan mrežasti obrazac, to je jasan indikator "blocking" artefakata, što potvrđuje da je nivo kvantizacije bio previše agresivan za dati 8x8 blok. Korigovanje kompresije se može izvršiti promjenom kvantizacione tabele, u kojoj manje vrijednosti donose manju kompresiju => sliku većeg kapaciteta=> manju grešku.

3. Optimizacije

U ovom poglavlju se iznose glavna sredstva optimizacije koda priloženog na github repozitorijumu. Sve optimizacije su navedene hronološki. Rezultati izvršavanja uključuju broj ciklusa izvršavanja kompresije čitave slike fiksne dimenzije za sve optimizacije kao i broj ciklusa izvršavanja "bottle-neck" dijela programa što je u tom trenutku bila funkcija *dct_and_quantize(...)*

1.

```
+ #pragma section("seg_block1")
+ signed short segments[120000];
+
+ #pragma section("seg_block2")
+ unsigned char data2[120000];
```

5. Raspored nizova

U prvoj verziji koda segmentiranje je praćeno centriranjem piksela, pri čemu je ova funkcija centriranja piksela napisana tako da radi na pikselima čitave ulazne slike.

Izmjenom funkcije centriranja piksela oko nule na takav način da se ona izvršava nad svakim blokom zasebno, dobija se znatna ušteda u memoriji jer ova modifikacija u potpunosti briše niz *segments* te obrada blokova počinje sa centriranjem piksela oko nule. Ovo je omogućilo učitavanje slika znatno većih dimenzija nauštrb vremenskih performansi.

2.

Druga značajna izmjena uključuje lokaciju u memoriji niza koji se koristi za obradu blokova. Na početku je to bio standardni blok inicijalizovan na sledeći način:

```
+   for (int block = 0; block < total_blocks; block++) {  
+       signed short* current_block = &segments[block * 64];
```

6. Pozicija *current_block* niza

Pomijeranjem inicijalizacije niza izvan glavne *main* funkcije te naznačavanjem da se radi o varijabli sa PM pristupom, ušteda memorije se svodi sa:

```
Loading application: "C:\Users\Korisnik\cces\2.12.1\jpeg_compression\Debug\jpeg_compression.dxe"  
Load complete.  
Number of total cycles for DCT and quantize: 111887115  
Number of total cycles for 131x113 image: 115502499
```

7. Prije postavljanja *current_block* u PM

na:

```
Loading application: "C:\Users\Korisnik\cces\2.12.1\jpeg_compression\Debug\jpeg_compression.dxe"  
Load complete.  
Number of total cycles for DCT and quantization: 86756100  
Number of total cycles for 131x113 image: 88642901
```

8. Nakon postavljanja *current_block* u PM

To je postignuto na sledeći način:

```
+ #pragma section("seg_pm_mem")  
+ #pragma align 4  
+ signed short pm current_block[64];
```

9. *Current_block*

Glavni razlog za ubrzanje je Dual-Data Fetch mogućnost SHARC arhitekture koja omogućava procesoru da dohvati dva podatka u jednom ciklusu, ali samo pod uslovom da se oni nalaze u različitim memorijskim blokovima (PM i DM pristup).

3.

```
+ void initialize_DMA()
+ {
+     *pAMICTL1 |= (AMIEN | BW16 | WS1);
+     *pMTMCTL   = MTMFLUSH;
+     asm("nop;nop;nop;nop;nop;");
+     *pMTMCTL   &= ~MTMFLUSH;
+     *pIIEP0     = (unsigned int)uncentered_block;
+     *pIMEP0     = 1;
+     *pICEP0     = 64;
+     *pEIEP0     = (unsigned int)data2;
+     *pEMEP0     = 1;
+     *pDMAC0     |= DEN;
+ }
```

10. Inicijalizacija DMA

Nakon premijestanja *current_block* niza u PM memoriju, osposobljen je DMA i to na sledeći način:

Osposobljavanje direktnog pristupa memoriji je eksterno-internog tipa. Glavni razlog za to jeste lokacija ulaznog bafera koji se nalazi u bloku *mem_sram* memorije i radnog bafera *uncentered_block* koji je lociran u data (DM pristup) memoriji. Izlazni bafer nije bilo moguće postaviti u programsku (PM tip) memoriju jer DMA može raditi samo sa DM magistralom.

DMA je korištena na sledeći način:

```
for (int block = 0; block < total_blocks; block++) {
    int index = block * 64;

    calculate_inv_qt(inv_qt);

    while(*pECEP0 > 0);

    if (expected_true(block < total_blocks - 1))
    {
        int next_buf = 1 - current_buf;
        int next_index = (block + 1) * 64;
        *pDMAC0 &= ~DEN;
        *pEIEP0 = (unsigned int)(data2 + next_index);
        *pIIEP0 = (unsigned int)&uncentered_block[next_buf*64];
        *pICEP0 = 64;
        *pDMAC0 |= DEN;
    }
    int uncentered_index = current_buf * 64;
```

11. DMA korištenje

Ovaj kod implementira tehniku dvostrukog baferovanja (Double Buffering) kroz direktnu manipulaciju DMA registrima, gde se procesor i DMA kontroler smjenjuju u radu nad dva različita dijela *uncentered_block* niza. Dok procesor obrađuje trenutni blok (označen sa *current_buf*), DMA u pozadini vrši prenos podataka za sledeći blok (*next_buf*) direktno iz spoljašnje memorije u procesorsku; linija *while(*pECEP0 > 0)* služi kao sigurnosni mehanizam koji zaustavlja procesor dok DMA ne završi prenos, nakon čega se parametri (poput spoljašnje adrese *pEIEP0* i unutrašnje *pIIEP0*) rekonfigurišu za naredni ciklus, a uloge bafera se zamenjuju komandom 1 - *current_buf*.

Ova implementacija DMA nije pokazala nikakva poboljšanja vremenskih performansi.

Glavni razlog zašto ovaj pristup nije donio značajna poboljšanja u brzini leži u tome što su, u tom trenutku, trenutne implementacije računskih operacija (DCT, kvantizacija, Huffman) u poređenju sa brzinom DMA prenosa previše spore, pa procesor provodi većinu vremena radeći, dok DMA završi prenos od 64 bajta gotovo trenutno. DMA donosi ogromne benefite samo kada su preuzimanje podataka i obrada vremenski približno jednaki; u ovom načinu korištenja, pošto je prenos 64 bajta trivijalan za SHARC kontroler, procesor "pretekne" DMA i skoro nikada ne čeka na while petlji, pa se vrijeme izvršavanja svodi na isto vrijeme izvršavanja kao i kad DMA nije bilo uvedeno. Uvođenje direktnog pristupa memoriji će pokazati benefite tek kad se smanji vrijeme izvršavanja "bottle-neck" dijela koda.

4.

S ciljem ubrzanja izvršavanja "bottle-neck" dijela programa, DCT koeficijenti se upisuju u LUT (look-up table) i time se eliminiše potreba za skupim izračunavanjem trigonometrijskih funkcija (*cos*) u realnom vremenu unutar najdubljih petlji algoritma. Standardne funkcije poput *cos()* zahtjevaju stotine procesorskih ciklusa jer se oslanjaju na aproksimacije, dok LUT pretvara taj problem u običnu operaciju čitanja iz memorije koja na SHARC arhitekturi traje samo jedan ciklus. Dodatno, pošto su trenutne tabele unaprijed izračunate i smještene u memoriju, kompajler može da koristi paralelne prenose podataka (Dual-

12. Broj ciklusa izvršavanja nakon dodavanja LUT

Data Fetch) i da istovremeno povlači i koeficijent iz tabele i piksel iz bloka, čime se procesorska snaga troši isključivo na matematiku transformacije umjesto na generisanje baze.

5.

Promjena sa direktne 4D implementacije na 2D (koja se izvodi kao dva prolaza od po 3D drastično poboljšava performanse zbog matematičke redukcije broja operacija koju omogućava separabilnost DCT-a.

```
Number of total cycles for DCT and quantization: 1157190
Number of total cycles for 131x113 image: 2840584
```

13. Broj ciklusa izvršavanja nakon promjene DCT izračunavanja

6.

Još jedan način koji odražava cijenu operacije dijeljenja nasuprot množenju se ogleda u modifikaciji vrijednosti kvantizacione tabele. Do ovog momenta su se njene vrijednosti u svakom koraku, za svaki element bloka koristile tako što se DCT koeficijent dijelio sa odgovarajućom vrijednošću kvantizacione tabele. Uvođenjem recipročne kvantizacione tabele te množenjem svakog DCT koeficijenta sa odgovarajućom recipročnom vrijednošću kvantizacione tabele dobija se velika ušteda. Glavni razlog leži u tome što se množenje može izvršiti paralelno, dok je dijeljenje inherentno iterativni proces koji se izvodi korak po korak.

```
Loop complete.
Number of total cycles for DCT and quantization: 874905
Number of total cycles for 131x113 image: 3132881
```

14. Broj ciklusa izvršavanja nakon uvođenja recipročne kvantizacione tabele

7.

Dalje je razdvajanjem DCT operacije i kvantizacije u dvije funkcije rezultovalo izolovanjem "bottle-neck" dijela programa. Sada analiza pokazuje tačnije rezultate:

```
Loop complete.
Number of total cycles for DCT: 646680
Number of total cycles for 131x113 image: 2912383
```

15. Broj ciklusa izvršavanja nakon razdvajanja DCT operacije i kvantizacije

8.

Uvođenje pokazivačke aritmetike ubrzava rad programa jer omogućava direktan pristup memorijskim adresama i eliminiše potrebu za kompleksnim izračunavanjem indeksa niza u svakoj iteraciji petlje. Kada se koristi indeksni pristup (npr. niz[i]), procesor pri svakom koraku mora da izvrši množenje indeksa sa veličinom tipa podatka i sabiranje sa baznom adresom, dok kod pokazivača procesor koristi post-inkrementalno adresiranje koje je hardverski integrisano u DAG (Data Address Generator) jedinicu SHARC procesora. To znači da se u istom ciklusu u kojem se podatak dohvata, adresa pokazivača automatski ažurira za sledeći korak bez trošenja dodatnih instrukcija ALU jedinice, čime se drastično smanjuje broj operacija unutar kritičnih sekvenci poput DCT funkcije. Uvođenje pokazivačke aritmetike je donijelo velike benefite:

```
-----  
total cycle count for DCT: 493425  
total cycle count: 2694055
```

16. Broj ciklusa izvršavanja nakon uvođenja pokazivačke aritmetike

9.

Dodatnim sitnim modifikacijama dobijamo konačan broj ciklusa za sliku dimenzija 131x113:

```
==== complete====  
total cycle count for DCT: 492660  
total cycle count: 2421591
```

17. Konačan broj ciklusa

Ovo vrijeme odgovara približno 1930 broju ciklusa po jednom 8x8 bloku.

4. Potencijalna poboljšanja

Prelazak sa dot product (skalarnog proizvoda) na row product pristup (gde se cio red transformiše odjednom) omogućava procesoru da maksimalno iskoristi širinu svoje magistrale i registre, umesto da se fokusira na pojedinačne elemente. Ovo potencijalno smanjuje broj instrukcija za upravljanje petljama i potencijalno omogućava ADSP-21489 procesoru da efikasnije implementira svoj SIMD režim.

Korišćenje 2D DMA prenosa omogućilo bi da se hardverski automatizuje segmentiranje slike na 8x8 blokove, eliminišući potrebu da procesor ručno kopira piksele u *current_block* bafer pomoću sporih for petlji. Standardni DMA vrši samo linearni prenos, ali 2D DMA na SHARC procesoru koristi parametre *stride* i *count* kako bi "preskakao" bajtove. Ovo omogućava da se čitav 8x8 blok pripremi u unutrašnjoj memoriji kao kompaktan niz u pozadini dok procesor obrađuje prethodni blok, čime se postiže savršen paralelizam i oslobađa procesorska snaga koja je ranije bila "zarobljena" u rutinama za reorganizaciju podataka.

2D DMA nije dostupan na trenutno korištenom SHARC procesoru.

5. Zaključak

Poređenje originalne i dekompresovane slike putem mape greške (Error Map) pruža uvid u prirodu gubitaka unutar JPEG pipeline-a:

- Lokalizacija gubitaka: Mapa jasno pokazuje da greška nije uniformna; ona je najizraženija na mjestima sa visokim frekvencijama, kao što su oštre ivice i sitni detalji, gde kvantizacija "odbacuje" najviše informacija.
- Vizuelni artefakti: Tamne regije mape potvrđuju da su površine sa blagim prelazima (niske frekvencije) rekonstruisane gotovo bez gubitaka, dok pojava pravilnih mrežastih šara (blocking artefakti) direktno ukazuje na granice 8x8 blokova gde je kvantizacija bila previše agresivna.
- Objektivna vs. Subjektivna mera: Dok nam MSE (Mean Squared Error) daje jednu brojčanu vrednost za cio proces, mapa greške nam omogućava da razumijemo kako se taj

šum manifestuje vizuelno, potvrđujući da ljudsko oko lakše oprašta gubitke u teksturama nego u oštrim konturama objekata.

Ovaj projekat uspešno demonstrira balans između stope kompresije i vizuelnog integriteta.